UT_11.3_Carga y descarga de datos

La carga y descarga de datos son procesos cruciales en la gestión de un almacén de datos de Amazon Redshift. La carga se refiere a la ingestión de datos de diversas fuentes en tablas de Redshift, mientras que la descarga es el proceso de exportación de datos de Redshift a almacenamiento externo o aplicaciones. En un escenario típico, como un almacén de datos basado en un sistema de pedidos, es posible que necesite cargar todos los datos del día anterior en lugar de pedidos individuales. Aunque los datos se pueden cargar utilizando sentencias `INSERT` estándar, los métodos de carga masiva son mucho más eficientes, dado el gran volumen de datos que suelen manejar los almacenes de datos. Del mismo modo, la descarga le permite exportar datos de forma masiva para utilizarlos en otras aplicaciones o herramientas de análisis. En este tema se exploran varios métodos para cargar datos en Amazon Redshift desde diferentes fuentes, así como para descargar datos a un almacenamiento externo, como Amazon S3.

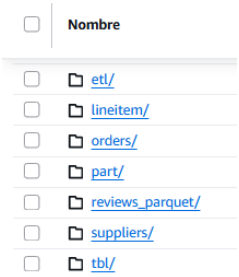
Existen múltiples formas de cargar datos en un almacén de datos de Amazon Redshift. La forma más común es utilizar el comando `COPY` para cargar datos desde Amazon S3. En este tema se tratarán todas las diferentes formas en que se pueden cargar datos en el almacén de datos de Amazon Redshift (sin servidor o clúster aprovisionado) desde diferentes fuentes.

En este tema se tratan los siguientes procedimientos:

1. Carga de datos desde Amazon S3 utilizando COPY
2. Actualización e inserción de datos
3. Catalogación e ingesta de datos con AWS Glue
4. Creación directa de base de datos en Data Catalog desde Redshift

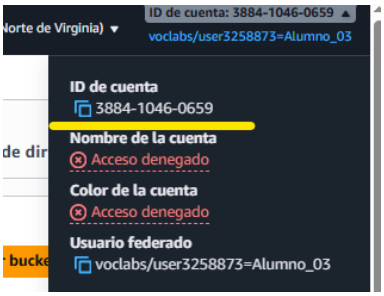
Requisitos técnicos

Estos son los requisitos técnicos para completar los procedimientos de este tema:



- Cree un *bucket* de Amazon S3 para almacenar temporalmente y descargar los datos en procedimientos específicos. Lo utilizaremos en las procedimientos como [`Su-Amazon_S3_Bucket`]. Crea dentro del mismo una estructura de carpetas como la de la imagen y guarda en ellas los archivos adjuntos al tema.

- Número de cuenta de AWS. Lo utilizaremos en los procedimientos como [`Your-AWS_Account_Id`].



En AWS Academy puedes encontrarlo en el menú superior derecho:

1. Carga de datos desde Amazon S3 utilizando COPY.

Amazon Redshift admite varias estructuras de modelos de datos, incluidas estructuras **dimensionales**, **desnormalizadas** y **agregadas** (*rollup*), lo que lo hace óptimo para el análisis.

En este procedimiento, configuraremos dos conjuntos de datos de muestra independientes en Amazon Redshift que están disponibles públicamente:

- Un modelo dimensional que utiliza un **esquema de referencia en estrella** (SSB) (`https://www.cs.umb.edu/~poneil/StarSchemaB.PDF`), un conjunto de datos basado en un sistema minorista.

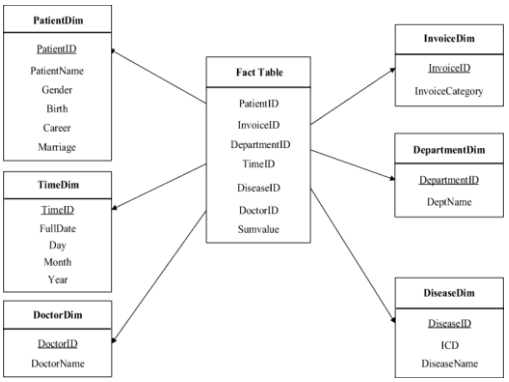
NOTA

Un Star Schema Benchmark (SSB) es un conjunto de tablas y consultas de prueba diseñado para evaluar el rendimiento de bases de datos analíticas (Data Warehouses), usando un modelo en esquema estrella (*star schema*).

¿Qué es un esquema estrella?

Un esquema estrella es un modelo típico de Data Warehouse donde:

- Hay una tabla de hechos (fact table) en el centro
- Varias tablas de dimensiones alrededor
- Las dimensiones se conectan directamente a la tabla central
- Visualmente forma una estrella



Para cargar los conjuntos de datos, utilizaremos el comando `COPY`, que permite copiar datos de Amazon S3 a un almacén de datos de Amazon Redshift (sin servidor o clúster aprovisionado), que es la forma recomendada de cargar datos de gran tamaño.

Cómo hacerlo...

Crearemos y cargaremos el siguiente modelo dimensional, basado en el **esquema estrella de referencia (SSB)** para un sistema minorista ilustrativo. Un esquema estrella es un modelo de datos dimensional en el que una tabla de hechos central contiene las principales métricas empresariales (hechos) y está rodeada por tablas de dimensiones (como los puntos de una estrella), de ahí su nombre.

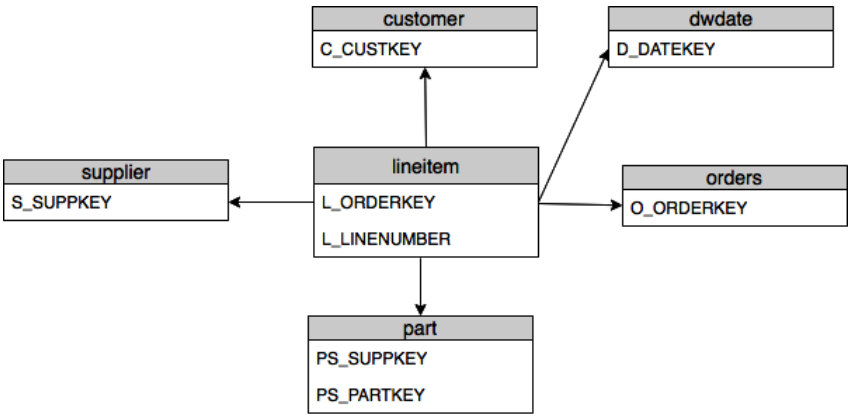


Figura 1: modelo de datos SSB

Ahora, creemos las tablas que imitan el modelo de datos anterior y rellenemos las tablas con los datos:

- Comenzaremos configurando los datos en un *bucket* de Amazon S3.
- Descargue el *script* SQL `Ssb_Table_Ddl.sql`, cópielo y péguelo en cualquier herramienta cliente SQL, y ejecútelo para crear el modelo dimensional para el conjunto de datos del sistema minorista:
  (El código completo de creación de tablas se encuentra en el archivo `Ssb_Table_Ddl.sql`)

```
DROP TABLE IF EXISTS lineitem;
DROP TABLE IF EXISTS supplier;
DROP TABLE IF EXISTS part;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS customer;
DROP TABLE IF EXISTS dwdate;
CREATE TABLE customer
```

```
(
  C_CUSTKEY      BIGINT NOT NULL,
  C_NAME         VARCHAR(25),
  C_ADDRESS      VARCHAR(40),
  C_NATIONKEY    BIGINT,
  C_PHONE        VARCHAR(15),
  C_ACCTBAL      DECIMAL(18,4),
  C_MKTSEGMENT   VARCHAR(10),
  C_COMMENT      VARCHAR(117)
…
CREATE TABLE dwdate
(
  d_datekey            INTEGER NOT NULL,
  d_date               VARCHAR(19) NOT NULL,
  d_dayofweek          VARCHAR(10) NOT NULL,
  d_month              VARCHAR(10) NOT NULL,
  d_year               INTEGER NOT NULL,
…
  d_lastdayinweekfl    VARCHAR(1) NOT NULL,
  d_lastdayinmonthfl   VARCHAR(1) NOT NULL,
  d_holidayfl          VARCHAR(1) NOT NULL,
  d_weekdayfl          VARCHAR(1) NOT NULL
);
```

Ahora cargaremos datos desde el *bucket* público de S3 a las tablas anteriores. Utilice cualquier herramienta cliente SQL y ejecute el siguiente comando sustituyendo los valores `[mibucket-redshift]`, `[Your-AWS_Account_Id]` y `[Your-Redshift_Role]`(en AWS Academy es LabRole) en el siguiente *script*. En cada comando `COPY`, observe que se especifican el formato de archivo y la compresión, como *csv* y *gzip*:

```
COPY customer
from 's3://mibucket-redshift/customer/'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'
CSV gzip;
COPY orders
from 's3://mibucket-redshift/orders/'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'
CSV gzip;
COPY part
from 's3://mibucket-redshift/part/'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'
CSV gzip;
COPY supplier
from 's3://mibucket-redshift/supplier/'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'
CSV gzip;
COPY lineitem
from 's3://mibucket-redshift/lineitem/'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'
```

```
CSV gzip;
COPY dwdate
from 's3://mibucket-redshift/dwdate/'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'
CSV gzip dateformat 'auto';
```

Nota

El *script* tardará unos minutos en completarse. Cada carga de tabla generará el mensaje `Load into table *** completed, *** record(s) loaded successfully` para confirmar que la ejecución se ha realizado correctamente.

Verifique que todas las tablas se hayan cargado con el número correcto de filas utilizando el siguiente comando y el resultado:

```
select count(1) from  lineitem;
select count(1) from  supplier;
select count(1) from  part;
select count(1) from  orders;
select count(1) from  customer;
select count(1) from  dwdate;
```

Ahora, el modelo dimensional está listo para realizar consultas. Podemos ejecutar una consulta analítica como la siguiente para unir las diferentes tablas del modelo dimensional y recuperar las métricas de pedidos para cada segmento de mercado en 1992:

```
SELECT c_mktsegment,
       COUNT(o_orderkey) AS orders_count,
       SUM(l_quantity) AS quantity,
       SUM(l_extendedprice) AS extendedprice,
       COUNT(DISTINCT P_PARTKEY) AS parts_count,
       COUNT(DISTINCT L_SUPPKEY) AS supplier_count,
       COUNT(DISTINCT o_custkey) AS customer_count
FROM lineitem
  JOIN orders ON l_orderkey = o_orderkey
  JOIN customer c ON o_custkey = c_custkey
  JOIN dwdate
    ON d_date = l_commitdate
   AND d_year = 1992
  JOIN part ON P_PARTKEY = l_PARTKEY
  JOIN supplier ON L_SUPPKEY = S_SUPPKEY
GROUP BY c_mktsegment;
```

Además del modelo dimensional, creemos también una tabla desnormalizada utilizando los datos de reseñas de productos de Amazon. Cree la tabla de datos de reseñas de productos utilizando:

```
CREATE TABLE product_reviews(
  marketplace varchar(2),
  customer_id varchar(32),
  review_id varchar(24),
  product_id varchar(24),
  product_parent varchar(32),
  product_title varchar(512),
  star_rating int,
  helpful_votes int,
  total_votes int,
  vine char(1),
```

```
  verified_purchase char(1),
  review_headline varchar(256),
  review_body varchar(max),
  review_date date,
  year int,
  product_category varchar(32)
)
DISTSTYLE KEY
DISTKEY (customer_id)
SORTKEY (
    marketplace,
    product_category,
    review_date);
```

Ahora, carguemos los datos de las reseñas en la tabla `product_reviews` ejecutando el siguiente comando en el cliente SQL:

```
COPY product_reviews

FROM 's3://mibucket-redshift/reviews_parquet/'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'

PARQUET;
```

Ahora, la tabla `product_reviews` está lista para realizar consultas, y puede ejecutar la siguiente consulta para obtener los 10 productos más votados en la categoría de productos `de ropa`:

```
SELECT product_title,
       SUM(total_votes)
FROM product_reviews
WHERE product_category = 'Apparel'
GROUP BY product_title
ORDER BY SUM(total_votes) DESC LIMIT 10;
```

Ahora, hemos utilizado Amazon S3 para trasladar datos a Amazon Redshift mediante el comando `COPY` y hemos configurado un conjunto de datos **dimensional** y **desnormalizado**.

Cómo funciona...

El comando **`COPY`** de Amazon Redshift se utiliza para cargar grandes conjuntos de datos en Amazon Redshift desde Amazon S3. Este es el enfoque recomendado, ya que el comando `COPY` aprovecha el **procesamiento masivamente paralelo** (**MPP**) de Amazon Redshift para ingestar los datos en la tabla de Amazon Redshift de manera eficiente. El comando `COPY` también ofrece varias formas de incorporar los archivos entrantes. Esto incluye la compatibilidad con múltiples formatos de archivo (como CSV, Parquet y JSON) con gestión de errores y la flexibilidad de incorporar todo tipo de datos estructurados.

Véase también...

Para obtener más información sobre el uso del comando `COPY` desde Amazon S3, visite **https://docs.aws.amazon.com/redshift/latest/dg/copy-parameters-data-source-s3.html**.

Eche un vistazo a las prácticas recomendadas para el comando `COPY` en **https://docs.aws.amazon.com/redshift/latest/dg/c_loading-data-best-practices.html**.

2. Actualización e inserción de datos

Un proceso **de extracción carga y transformación** (**ELT**) es una técnica común para actualizar el almacén de datos desde el sistema de origen. El proceso ELT se puede ejecutar como un proceso por lotes casi en tiempo real que permite preparar los datos del sistema de origen y

realizar una actualización masiva en el almacén de datos de Amazon Redshift (sin servidor o clúster aprovisionado). Amazon Redshift, al ser un sistema RDBMS, permite la actualización de datos en forma de operaciones `MERGE`, que combinan la funcionalidad de las operaciones `INSERT`, `UPDATE` y `DELETE`. En este procedimiento, profundizaremos en algunas de las estrategias ELT comunes para actualizar un modelo dimensional utilizando el comando `MERGE`.

Preparación

Inicie sesión en el almacén de datos Amazon Redshift utilizando un cliente SQL o el editor de consultas v2 y cree la tabla `part`:

```
DROP TABLE IF EXISTS part;

CREATE TABLE part
(
  P_PARTKEY       BIGINT NOT NULL,
  P_NAME          VARCHAR(55),
  P_MFGR          VARCHAR(25),
  P_BRAND         VARCHAR(10),
  P_TYPE          VARCHAR(25),
  P_SIZE          INTEGER,
  P_CONTAINER     VARCHAR(10),
  P_RETAILPRICE   DECIMAL(18,4),
  P_COMMENT       VARCHAR(23)
)
diststyle ALL;
```

Ejecute el comando `COPY` para cargar en la tabla `part` desde S3:

```
COPY part
FROM 's3:// mibucket-redshift /tbl/part.tbl'
iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift_Role]'
DELIMITER '|'
IGNOREHEADER 0
REMOVEQUOTES
TRIMBLANKS
EMPTYASNULL
BLANKSASNULL
REGION 'us-east-1';
```

NOTA

DELIMITER '|'
Define el separador de columnas.

IGNOREHEADER 0
Indica cuántas filas iniciales ignorar.

- 0 = no ignora ninguna fila.
- 1 = ignora la primera (cuando hay encabezados).

REMOVEQUOTES

Elimina comillas que envuelven los campos.
Ejemplo:
`"Juan"|"Ventas"|100`
Se cargará como:
`Juan | Ventas | 100`

TRIMBLANKS
Elimina espacios en blanco al inicio y final de los campos tipo VARCHAR.
Ejemplo:
`" Juan "`
Se convierte en:
`Juan`

EMPTYASNULL
Convierte *strings* vacíos ('') en NULL.
Ejemplo:
`||`
Si el campo está vacío → se guarda como `NULL`.

BLANKSASNULL
Convierte campos que contienen **solo espacios** en NULL.
Ejemplo:
`|  |`
Se interpreta como `NULL`.

Cómo hacerlo...

Este procedimiento ilustra cómo actualizar la dimensión `part`, seguida de la tabla de hechos `de lineitem`. Las tablas dimensionales se actualizarán primero y, a continuación, la tabla de hechos para mantener la integridad de los datos. Comencemos con la actualización de datos para la dimensión `part`:

Abra cualquier herramienta cliente SQL e inicie la transacción para actualizar la tabla de dimensiones `part`:

```
BEGIN TRANSACTION;
```

Consejo

El uso de la transacción para actualizar los datos permite revertir el proceso si se produce un error y, además, los usuarios finales no ven el estado intermedio del cambio de datos.

Cree la tabla provisional y cargue los datos incrementales entrantes desde la fuente:

```
/* Cree una tabla provisional para almacenar los datos de entrada. La tabla
provisional se crea con la opción BACKUP NO para acelerar las inserciones y
también porque los datos son temporales */
DROP TABLE IF EXISTS stg_part;
CREATE TABLE stg_part
(
  NAME          VARCHAR(55),
  MFGR          VARCHAR(25),
  BRAND         VARCHAR(10),
  TYPE          VARCHAR(25),
  SIZE          INTEGER,
  CONTAINER     VARCHAR(10),
  RETAILPRICE   DECIMAL(18,4),
  COMMENT       VARCHAR(23)
)
BACKUP NO
;
```

```
COPY stg_part
FROM 's3://mibucket/etl/part/' iam_role 'arn:aws:iam::[Your-
AWS_Account_Id]:role/LabRole'csv gzip compupdate preset;
```

NOTA

csv
Indica que el formato del archivo es CSV (Comma Separated Values).
Si no se especifica DELIMITER, el separador por defecto es ,.

gzip
Indica que los archivos están comprimidos en formato GZIP (.gz).
Redshift:
- Descomprime automáticamente
- Luego carga los datos

Esto mejora:
- Velocidad de transferencia
- Uso de almacenamiento en S3

compupdate preset
Controla cómo Redshift maneja la compresión de columnas.

compupdate
Le dice a Redshift que gestione la compresión automáticamente.

preset
Usa el esquema de compresión ya definido en la tabla (no recalcula uno nuevo).

Consejo

Tenga en cuenta que los datos incrementales del día `2020-08-15` se cargan en la tabla `stg_part`.

Los datos se fusionarán en la tabla de dimensiones de piezas utilizando el comando `MERGE`, que actualizará los registros coincidentes existentes e insertará nuevos registros. Las operaciones de actualización e inserción se realizan basándose en la clave natural de la tabla:

```
-- 1 Actualizar registros existentes
UPDATE part
SET
    p_mfgr = stg_part.mfgr,
    p_brand = stg_part.brand,
    p_type = stg_part.type,
    p_size = stg_part.size,
    p_container = stg_part.container,
    p_retailprice = stg_part.retailprice,
    p_comment = stg_part.comment
FROM stg_part
WHERE part.p_name = stg_part.name;

-- 2 Insertar registros nuevos
INSERT INTO part (
    p_name, p_mfgr, p_brand, p_type, p_size,
    p_container, p_retailprice, p_comment
)
SELECT
    stg_part.name,
    stg_part.mfgr,
    stg_part.brand,
    stg_part.type,
    stg_part.size,
    stg_part.container,
```

```
    stg_part.retailprice,
    stg_part.comment
FROM stg_part
LEFT JOIN part
    ON part.p_name = stg_part.name
WHERE part.p_name IS NULL;
```

Lo habitual sería usar una sentencia MERGE como la de abajo, pero esto no funciona en la versión de nuestro clúster.

```
MERGE INTO part AS tgt
USING stg_part AS src
ON tgt.p_name = src.name
WHEN MATCHED THEN
  UPDATE SET
    p_mfgr = src.mfgr,
    p_brand = src.brand,
    p_type = src.type,
    p_size = src.size,
    p_container = src.container,
    p_retailprice = src.retailprice,
    p_comment = src.comment
WHEN NOT MATCHED THEN
  INSERT (p_partkey, p_name, p_mfgr, p_brand, p_type, p_size, p_container,
p_retailprice, p_comment)
  VALUES (
    (SELECT MAX(p_partkey) + ROW_NUMBER() OVER (ORDER BY src.name) FROM part),
    src.name,
    src.mfgr,
    src.brand,
    src.type,
    src.size,
    src.container,
    src.retailprice,
    src.comment
  );
```

La actualización de datos ya se ha completado en la dimensión `de pieza` de destino. Confirme la transacción con el siguiente comando:

```
--  commit and End transaction
END TRANSACTION;
```

Nota

Del mismo modo, puede repetir los pasos anteriores para otras tablas dimensionales antes de iniciar la tabla de hechos.

Ahora, actualicemos la tabla de hechos `lineitem` utilizando el siguiente *script*. Inicie la transacción para la tabla de hechos `lineitem`:

```
-- Iniciar una nueva transacción
BEGIN TRANSACTION;
```

Cree la tabla provisional para almacenar los datos incrementales entrantes, como se muestra en el siguiente bloque:

```
-- Drop stg_lineitem if exists
DROP TABLE IF EXISTS stg_lineitem;
-- Create a stg_lineitem staging table and COPY data from input S3 location with
the refreshed incremental data
```

```
CREATE TABLE stg_lineitem(
  orderkey        BIGINT,
  LINENUMBER      INTEGER NOT NULL,
  QUANTITY        DECIMAL(18,4),
  EXTENDEDPRICE   DECIMAL(18,4),
  DISCOUNT        DECIMAL(18,4),
  TAX             DECIMAL(18,4),
  RETURNFLAG      VARCHAR(1),
  LINESTATUS      VARCHAR(1),
  SHIPDATE        DATE,
  COMMITDATE      DATE,
  RECEIPTDATE     DATE,
  SHIPINSTRUCT    VARCHAR(25),
  SHIPMODE        VARCHAR(10),
  COMMENT         VARCHAR(44),
  p_name          VARCHAR(55),
  s_name          VARCHAR(25)
)
BACKUP NO;

COPY stg_lineitem
FROM 's3://mibucket/etl/lineitem/' iam_role 'arn:aws:iam::[Your-
AWS_Account_Id]:role/LabRole'csv gzip;
```

Consejo

Tenga en cuenta que los datos incrementales para `2020-08-15` se cargan en la tabla `stg_lineitem`.

Elimine cualquier dato existente (si lo hay) para `el 15/08/2020` y actualícelo con los datos actuales para esta fecha:

```
-- Se elimina cualquier fila de destino para la fecha de entrada para
idempotencia
DELETE FROM lineitem WHERE l_shipdate = '2020-08-15';
```

Combine los nuevos datos entrantes para `el 15-18-2020` en la tabla de hechos `lineitem` utilizando la siguiente instrucción:

```
BEGIN;

DELETE FROM lineitem
USING (
    SELECT
        src.orderkey,
        p.p_partkey,
        s.s_suppkey
    FROM stg_lineitem src
    JOIN part p ON p.p_name = src.p_name
    JOIN supplier s ON s.s_name = src.s_name
    WHERE src.shipdate = '2020-08-15'
) d
WHERE lineitem.l_orderkey = d.orderkey
AND lineitem.l_partkey = d.p_partkey
AND lineitem.l_suppkey = d.s_suppkey;

INSERT INTO lineitem
SELECT
    src.orderkey,
    p.p_partkey,
    s.s_suppkey,
    src.linenumber,
    src.quantity,
```

```
    src.extendedprice,
    src.discount,
    src.tax,
    src.returnflag,
    src.linestatus,
    src.shipdate,
    src.commitdate,
    src.receiptdate,
    src.shipinstruct,
    src.shipmode,
    src.comment
FROM stg_lineitem src
JOIN part p ON p.p_name = src.p_name
JOIN supplier s ON s.s_name = src.s_name
WHERE src.shipdate = '2020-08-15';

END;
```

La sentencia MERGE equivalente sería:

```
WITH supplier_dim AS
  (SELECT DISTINCT s_name, s_suppkey FROM supplier),
part_dim AS
  (SELECT DISTINCT p_name, p_partkey FROM part)
MERGE INTO lineitem AS tgt
USING stg_lineitem AS src
  ON tgt.l_shipdate = '2020-10-15'
    AND tgt.l_partkey = (SELECT p_partkey FROM part_dim WHERE p_name = src.p_name)
    AND tgt.l_suppkey = (SELECT s_suppkey FROM supplier_dim WHERE s_name = src.s_name)
WHEN NOT MATCHED THEN
  INSERT (
    l_orderkey, l_partkey, l_suppkey, l_linenumber, l_quantity,
    l_extendedprice, l_discount, l_tax, l_returnflag, l_linestatus,
    l_shipdate, l_commitdate, l_receiptdate, l_shipinstruct, l_shipmode, l_comment
  )
  VALUES (
    src.orderkey, (SELECT p_partkey FROM part_dim WHERE p_name = src.p_name),
    (SELECT s_suppkey FROM supplier_dim WHERE s_name = src.s_name),
    src.linenumber, src.quantity, src.extendedprice, src.discount, src.tax,
    src.returnflag, src.linestatus, src.shipdate, src.commitdate, src.receiptdate,
    src.shipinstruct, src.shipmode, src.comment);
```

Nota

Tenga en cuenta que las claves dimensionales se derivan de la tabla dimensional utilizando las claves naturales.

La actualización de todos los datos ya se ha completado en la tabla de hechos `lineitem` de destino. Confirme la transacción utilizando el siguiente código:

```
--  confirmar y finalizar transacción
COMMIT;
```

Nota

Tenga en cuenta que todos los datos de las tablas de dimensiones y hechos se gestionan de forma masiva para actualizar/insertar todos los datos entrantes de una sola vez. Se trata de una práctica recomendada, ya que el esfuerzo necesario para realizar **el lenguaje de manipulación de datos (DML)** en unas pocas filas o en varias filas es prácticamente el mismo.

Ahora dispone de un modelo dimensional actualizado con los datos más recientes, que puede verificar ejecutando la siguiente consulta:

```
SELECT c_mktsegment,
```

```
       COUNT(o_orderkey) AS orders_count,
       SUM(l_quantity) AS quantity,
       SUM(l_extendedprice) AS extendedprice,
       COUNT(DISTINCT P_PARTKEY) AS parts_count,
       COUNT(DISTINCT L_SUPPKEY) AS supplier_count,
       COUNT(DISTINCT o_custkey) AS customer_count
FROM lineitem
  JOIN orders ON l_orderkey = o_orderkey
  JOIN customer c ON o_custkey = c_custkey
  JOIN part ON P_PARTKEY = l_PARTKEY
  JOIN supplier ON L_SUPPKEY = S_SUPPKEY
WHERE l_shipdate = '2020-08-15'
GROUP BY c_mktsegment;
```

La estrategia ELT anterior ahora se puede integrar con cualquier herramienta de flujo de trabajo que permita la actualización automática del almacén de datos.

3. Catalogación e ingesta de datos con AWS Glue

Los datos almacenados en Amazon S3 se pueden catalogar utilizando el servicio AWS Glue. La catalogación de los datos permite adjuntar los metadatos y rellenar el catálogo de datos de AWS Glue. Este proceso enriquece los datos sin procesar que se pueden consultar como tablas utilizando muchos de los servicios analíticos de AWS, como Amazon Redshift o Amazon EMR, para el procesamiento analítico. Es fácil realizar este descubrimiento de datos utilizando los rastreadores de AWS Glue, que pueden crear y actualizar los metadatos automáticamente.

En este procedimiento, enriqueceremos los datos para catalogarlos y permitir su ingestión en un almacén de datos de Amazon Redshift (sin servidor o clúster aprovisionado).

Preparación

Para completar este procedimiento, necesitará:

- Un *bucket* de Amazon S3 para almacenar temporalmente y descargar los datos en procedimientos específicas, al que haremos referencia en los procedimientos como `[Your-Amazon_S3_Bucket].` El *bucket* de Amazon S3 debe estar en la misma región que su almacén de datos Amazon Redshift.
- El número de cuenta de AWS, al que haremos referencia en los procedimientos como `[Your-AWS_Account_Id]`.

Cómo hacerlo…

Este procedimiento catalogará e ingestará el conjunto de datos de reseñas de productos de clientes de Amazon.com en Amazon Redshift:

- Vaya a la consola de AWS y seleccione **AWS Glue**. Compruebe que se encuentra en la misma región de AWS que el almacén de datos de Amazon Redshift. En el menú de navegación izquierdo, dentro de AWS Glue, seleccione **Catálogo de datos** y, a continuación, seleccione **Rastreadores (Crawlers)**.
- Haga clic en el botón **Create Crawler** (**Crear rastreador**), escriba cualquier nombre de rastreador, como «`resenas`», y haga clic en **Next (Siguiente)**.
- En la página ***Choose data sources and classifiers*** (Elegir fuentes de datos y clasificadores), siga las instrucciones que se muestran en las capturas de pantalla *de la figura 2*. En la sección ***Data source configuration*** (Configuración de la fuente de datos), en la pregunta ***Is your data already mapped to Glue tables?*** **(¿Sus datos ya están asignados a tablas de Glue?**), seleccione ***Not yet*** (Todavía no) y, a continuación, elija ***Add a data source*** **(Añadir una fuente de datos)**. En el cuadro de diálogo que aparece, introduzca la siguiente información:

1. En Origen de datos, seleccione S3.
2. Para la ubicación de los datos S3, puede indicar si los datos se encuentran en la misma cuenta o en una cuenta diferente.
3. En Ruta S3, copie y pegue la ruta `s3:// mibucket/reviews_parquet/`.
4. En la opción **Ejecuciones posteriores del rastreador**, puede elegir cómo se comportará el rastreador. Para este tutorial, seleccione **Rastrear todas las subcarpetas** y seleccione **Añadir una fuente de datos S3**.
5. De vuelta en la página **Elegir fuentes de datos y clasificadores**, en la sección **Fuentes de datos**, verá la fuente de datos S3 que ha añadido. Seleccione **Siguiente** en esta página.

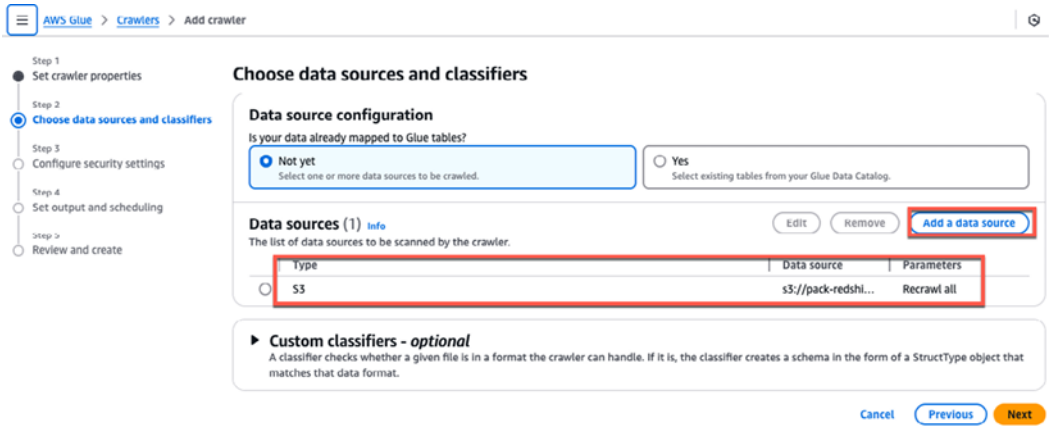


Figura 2: Añadir un rastreador

- En la página **Configurar ajustes de seguridad**, en **IAM role**, elija ***LabRole***, y haga clic en el botón **Siguiente**.
- En la página **Establecer salida y programación**, en la sección **Configuración de salida**, en **Base de datos de destino**, haga clic en **Añadir base de datos**, asigne a la base de datos un nombre representativo, como `reviews`, y haga clic en **Crear base de datos**.
- Vuelva a la página **Establecer salida y programación**, seleccione el botón Actualizar junto al menú desplegable **Base de datos de destino**, seleccione `reviews` y, en **Prefijo del nombre de la tabla**, introduzca `src_`. Por último, haga clic en **Siguiente**.

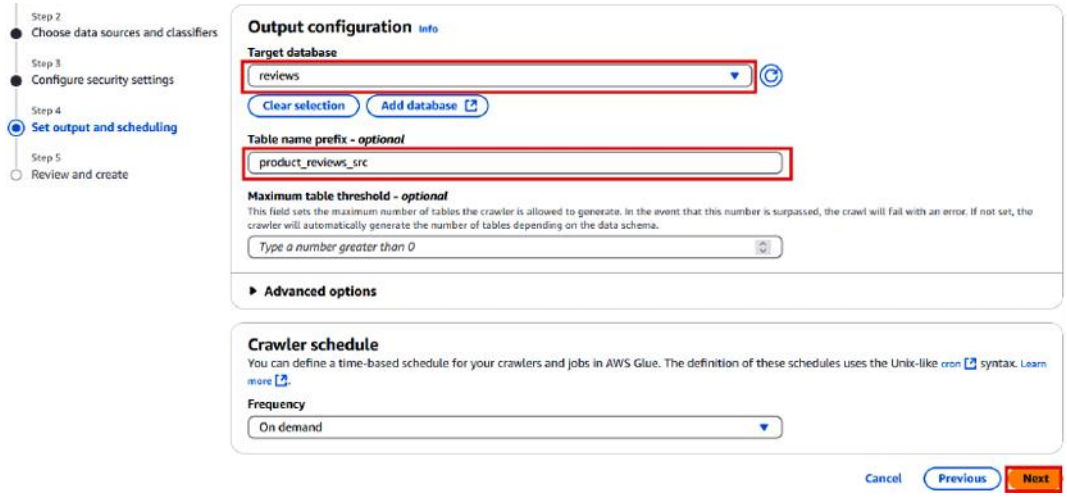


Figura 3: Configurar la salida del rastreador

En la página **Revisar y crear**, haga clic en **Crear rastreador** para crear el rastreador. En la página de detalles del rastreador, seleccione **Ejecutar rastreador** y espere hasta que el estado cambie a **Éxito**:

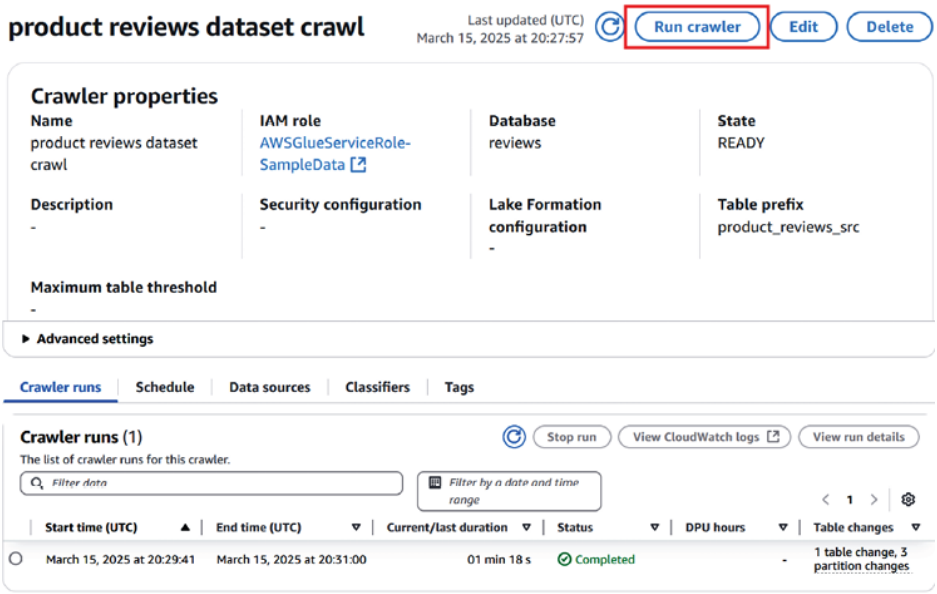


Figura 4: Supervisar el estado del rastreador

Ahora, AWS Glue ha rastreado el conjunto de datos de reseñas de productos y ha descubierto la tabla automáticamente. Puede verificar la tabla navegando a la opción **Tablas** para ver la tabla `src_reviews_parquet` en la lista:

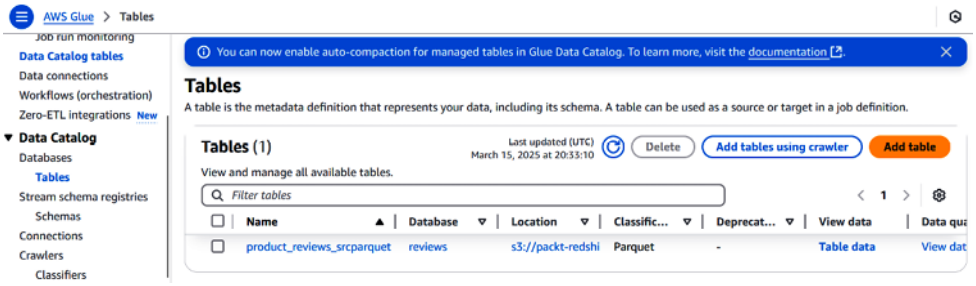
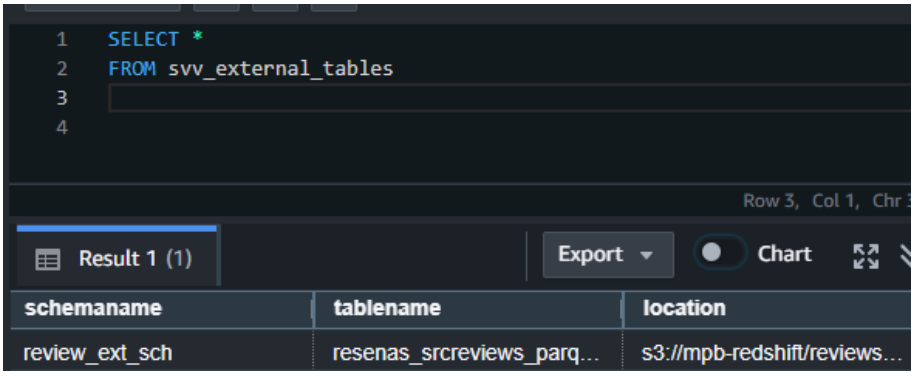


Figura 5: Ver la tabla creada por el rastreador

Abra cualquier herramienta cliente SQL, conéctese a Amazon Redshift y cree un esquema que apunte a la base de datos del catálogo de AWS Glue `de reseñas` utilizando el siguiente comando y sustituyendo los valores `[Your-AWS_Account_Id]` y `[Your-Redshift_Role]`:

```
CREATE external SCHEMA review_ext_sch
FROM data catalog
DATABASE 'reviews' iam_role 'arn:aws:iam::[Your-AWS_Account_Id]:role/[Your-Redshift-Role]'
```

Con el comando de la imagen pueden verse todas las tablas externas y la ubicación de los ficheros en S3:



Cree la tabla `product_reviews_stage` que almacenará los datos rastreados entrantes:

```
CREATE TABLE product_reviews_stage
(
  marketplace         VARCHAR(2),
  customer_id         VARCHAR(32),
  review_id           VARCHAR(24),
  product_id          VARCHAR(24),
  product_parent      VARCHAR(32),
  product_title       VARCHAR(512),
  star_rating         INT,
  helpful_votes       INT,
  total_votes         INT,
  vine                CHAR(1),
  verified_purchase   CHAR(1),
  review_headline     VARCHAR(256),
  review_body         VARCHAR(MAX),
  review_date         DATE,
  YEAR                INT
)
DISTSTYLE KEY DISTKEY (customer_id) SORTKEY (review_date);
```

Ahora, insertemos los datos de la tabla AWS Glue rastreada en la tabla Amazon Redshift, `product_reviews_stage`:

```
INSERT INTO product_reviews_stage
(
  marketplace,
  customer_id,
  review_id,
  product_id,
  product_parent,
  product_title,
  star_rating,
  helpful_votes,
  total_votes,
  vine,
  verified_purchase,
  review_headline,
  review_body,
  review_date,
  year
)
SELECT marketplace,
       customer_id,
       review_id,
       product_id,
       product_parent,
       product_title,
       star_rating,
       helpful_votes,
       total_votes,
       vine,
       verified_purchase,
       review_headline,
       review_body,
       review_date,
       year
FROM review_ext_sch.src_reviews_parquet;
```

Ahora, la tabla `public.product_reviews_stage` está lista para almacenar el conjunto de datos entrante, lo cual se puede verificar utilizando el siguiente comando:

```
SELECT * FROM product_reviews_stage;
```

Cómo funciona...

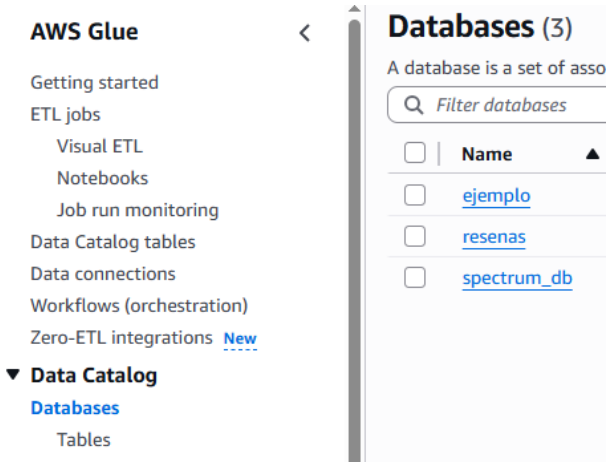
AWS Glue proporciona un rastreador que puede determinar automáticamente la estructura de los datos en Amazon S3. AWS Glue mantiene el catálogo de metadatos al que se puede acceder a través de otros servicios analíticos de AWS, como Amazon Redshift. Amazon Redshift puede leer/escribir datos en Amazon S3 directamente utilizando la función Amazon Redshift Spectrum.

4. Creación directa de base de datos en Data Catalog desde Redshift

De forma análoga, desde Redsifit utilizando Spectrum podemos crear directamente bases de datos y tablas en ellas sin necesidad de usar AWS Glue. En primer creamos un esquema que apunta a Data Catalog y una base de datos externa dentro de él, **`spectrum_db`** en nuestro caso

```
CREATE EXTERNAL SCHEMA spectrum_schema
FROM DATA CATALOG
DATABASE 'spectrum_db'
IAM_ROLE 'arn:aws:iam::<aws-account-id>:role/<your-role-name>'
CREATE EXTERNAL DATABASE IF NOT EXISTS;
```

Si desde AWS Glue vamos a consultar el Data Catalog veremos que se ha creado la base de datos allí:

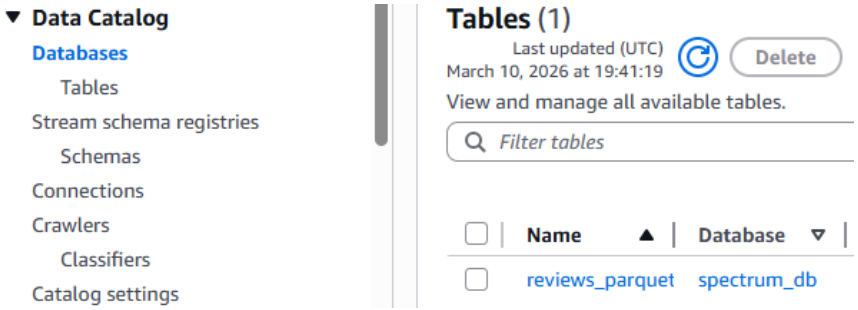


Nos quedaría creas la base de datos externa (en el ejemplo usaremos los mismos archivos *parquet* de reseñas que en el ejemplo anterior) enlazada al archivo de nuestro *bucket* S3.

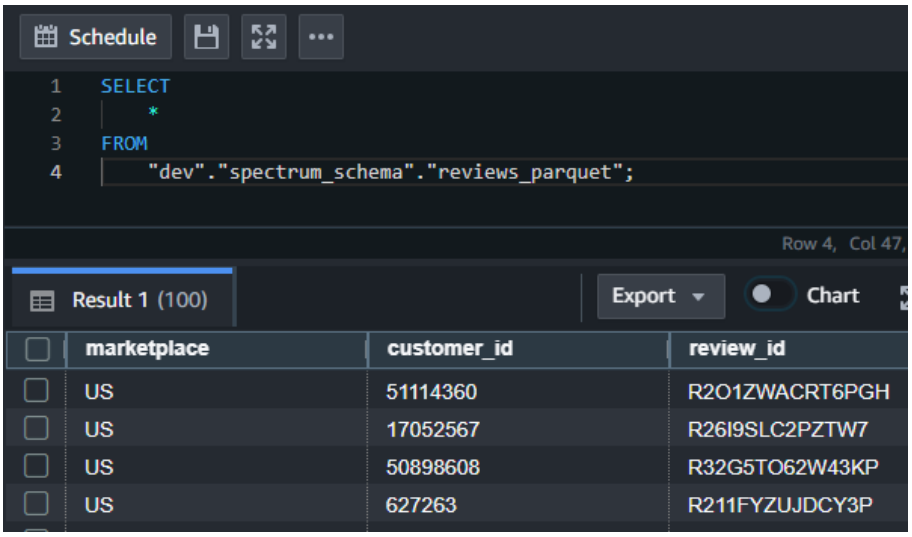
```
CREATE EXTERNAL TABLE spectrum_schema.reviews_parquet
(
  marketplace          VARCHAR(2),
  customer_id          VARCHAR(32),
  review_id            VARCHAR(24),
  product_id           VARCHAR(24),
  product_parent       VARCHAR(32),
  product_title        VARCHAR(512),
  star_rating          INT,
  helpful_votes        INT,
  total_votes          INT,
  vine                 CHAR(1),
  verified_purchase    CHAR(1),
  review_headline      VARCHAR(256),
  review_body          VARCHAR(MAX),
```

```
  review_date          DATE,
  YEAR                 INT
)
STORED AS PARQUET
LOCATION 's3:// your-bucket-name/reviews_parquet/';
```

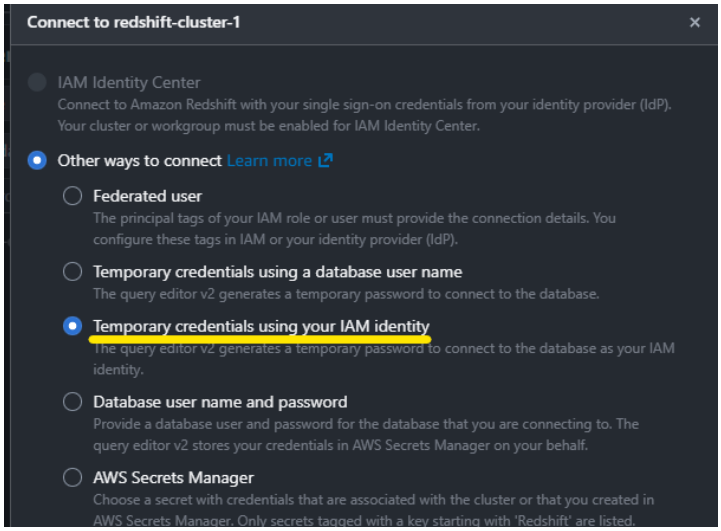
Vista de la table en el Data Catalog:



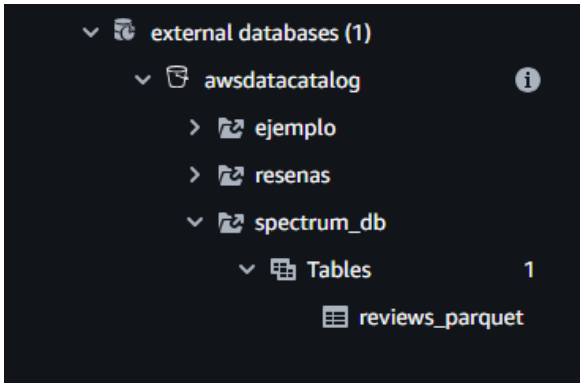
Consulta desde la consola de Redshift:



Si nos conectamos al clúster de como nuestro usuario IAM en lugar del propio que hemos definido para Redshift:



Podremos acceder también al Data Catalog y ver la base de datos y tabla que acabamos de crear.


external databases (1)
awsdatacatalog
ejemplo
resenas
spectrum_db
Tables
1
reviews_parquet